AGENT LAB

A fully-local AI personal assistant on Windows

What started it, what was built, how to copy it,
and where it stopped

By MD Fahim Nafis

Fahim | github.com/fahimnafis2025 | linkedin.com/in/fahimnafis

Status. Status: ARCHIVED and ABANDONED on 29 October 2026. This document is a historical record, not a maintained product and not a promise that the code will work unchanged on your machine. If you pick it up, start from Chapter 8 and Chapter 12, and test live, one step at a time.

Project dates in this file: 6 October 2026 to 29 October 2026.
Final files in the uploaded archive: agent.py, 9,538 lines; panel.py, 1,230 lines, footer V22.9.2.
Archive analysed for this document: agent-lab-2.zip, 328 files, excluding .venv and the voice-sidecar folder, which were too large to include.

Contents

This book is written for a stranger who finds this repository in the future and wants to understand it without asking the author. It explains the idea in plain language first, then the exact steps, then the honest results. Diagrams are included so you can see the shape of the system before you read the code.

- 1. The idea in one page
- 2. What Fahim wanted, and the rules he set
- 3. The machine and the software it ran on
- 4. Architecture: the head chef and the waiter
- 5. What is in the archive, file by file
- 6. Timeline: from first lab to final night
- 7. How a turn works, step by step
- 8. How to copy this work (reproduction guide)
- 9. The voice lab: the last big push
- 10. What worked, what did not, with evidence
- 11. Where it stopped, in the author's own terms
- 12. If you pick this up: what to do differently
- Appendix A. File inventory and counts
- Appendix B. Tool families in agent.py
- Appendix C. Model dock
- Appendix D. Safety, secrets, and limits

How to read this book. A note on numbers. Line counts, file counts, chat counts, and job counts in this book were counted from the uploaded archive on 29 October 2026. Speeds, VRAM estimates, and model sizes that were never measured on Fahim's PC are labelled as estimates wherever they appear. Do not quote an estimate as a measurement.

1. The idea in one page

Fahim wanted a personal assistant that lived on his own computer, not in someone else's cloud. It should answer questions, open programs, browse the web, remember facts, run background jobs, and eventually talk like a person, without sending his files, his history, or his voice to a paid service every time he asked something.

The shortcut would have been to rent a frontier model in the cloud. He refused that shortcut on purpose. The rules were local first and zero cost: if a step needed a paid API key, it was a proposal to approve, never an assumption. That rule shaped every design choice that follows, including the hardest ones at the end, when voice had to run on the CPU because the graphics card was already full with the brain.

In one sentence: a small, private, on-machine assistant that tries to be accurate before it tries to be fast, breaks a request into tickets, gives each ticket to a specialist worker, checks the worker's evidence, and only then gives the user an answer.

Short cut. If you only read one chapter, read Chapter 4 for the shape, Chapter 8 for how to run it, and Chapter 11 for why it stopped. Those three together give you the project without the detail.

2. What Fahim wanted, and the rules he set

2.1 The goal

Build a fully-local personal assistant on Windows that can do real work: files, web, desktop programs, background jobs, memory, and later, voice conversation. The assistant should be useful every day, not just a demo. Fahim said, in his own words during the build, that he never said he wanted it to be fast. He wanted it to be accurate.

2.2 The standing rules

Rule	What it meant in practice
Accuracy over speed	Cut wasted work, never cut verification. A fast wrong answer was worse than a slow honest one.
No hardcoding	Do not fix a conversational failure with a list of banned phrases. Fix the general mechanism, so it works for every prompt, not just the test prompt. Safety blocklists were the exception.
Local first, zero cost	No paid API by default. A paid or cloud move had to be proposed and approved first. This veto killed several otherwise attractive options.
Never copy a shared idea wholesale	A reel, link, or example was suggestion mode only. Dissect it, verify it against live sources, adopt only the useful parts, and reshape it for a 14-billion-parameter brain.
Approval is a card, not prose	The agent asked permission only for PowerShell or command-line use, and for serious writes: delete, install, publish, send, change system settings, or force-kill. The user's explicit order was permission for the action ordered. Everything else, reads and non-destructive tools, acted without asking.
Real capability before beautification	A working feature came first. A pretty panel that could not do the job was not progress.
Sandbox proof is not live proof	A test that ran in a sandbox with mocked audio or models proved the code compiled and the logic held. It did not prove it worked on Fahim's PC. Live proof needed the footer version, a real run, and a real result.
No malware, ever	The agent never wrote remote-code-execution or malware code. That refusal lived in the routing layer, before the model. Defensive security reviews, scanning code for flaws, stayed fully allowed.

2.3 What success would have looked like

One reliable daily capability, proven live, before any talk of giving the agent to other people. Not a list of features, not a demo video, but one job the user could trust every day. That bar was never met, and Fahim was honest about it during the build: he judged the agent, at that stage, as a show project, buggy and unreliable at real work, not yet able to do long research like a frontier assistant. This book keeps that judgement, because it explains the ending.

3. The machine and the software it ran on

Item	Detail in the archive
Computer	Windows PC, Intel i7-14700K, 32 GB DDR5 RAM

Item	Detail in the archive
Graphics card	NVIDIA RTX 4070 SUPER, 12 GB VRAM. This is the hard ceiling for the whole design.
Project folder	E:\AGENT\agent-lab, on the E: drive
Model storage	E:\AGENT\ollama-models, via the OLLAMA_MODELS setting
Model cache for voice	E:\AGENT\hf-cache
Main brain (default)	qwen3:14b through Ollama, about 9.3 GB in the dock manifest
Brain left in config at the end	huihui_ai/qwen3-abliterated:8b, in dock/config.json. This was left over from a test and had not been switched back. See Chapter 11.
Context at the end	16,384 tokens, set by Fahim on 29 October 2026. He reported speed felt fine at 16K. The build files still carried 8192, so how 16K was set, slider or file edit, matters for reinstalls. See Chapter 8.
Panel address	http://127.0.0.1:5000, Flask, threaded
Voice sidecar address	http://127.0.0.1:8002, FastAPI with uvicorn, localhost only
Panel style	BREACH TERMINAL, gold on dark, footer version string is the install proof



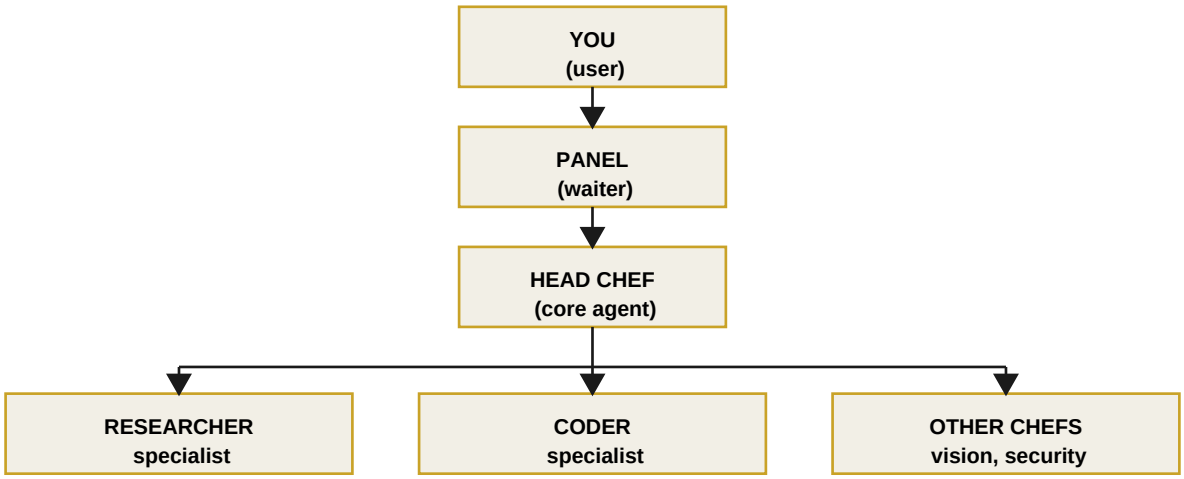
* KV numbers are estimates, not measured on this PC. 16K total is about 12.6 GB, so a small spill to RAM is expected. That is why voice runs on CPU: it must not take VRAM from the brain.

Figure 1. VRAM budget at 16K context. The brain alone is most of the card. That is why voice, vision, and other helpers could not also live on the GPU at the same time.

Hardware note. Do not copy the hardware numbers as requirements for a different machine. They are the machine this project was built and tested on. A smaller card will need smaller models or more offloading to RAM, and will be slower. A larger card will change the trade-offs in Chapter 9.

4. Architecture: the head chef and the waiter

Fahim prescribed this shape himself, with a restaurant analogy, on 12 October 2026. He said the current single-brain agent was not even a minimum viable product, because one brain doing everything was never what he wanted. The fix was orchestration, not a bigger prompt.



Workers report back. Head chef verifies, then plates the answer.

Figure 2. Head-chef architecture. The user talks to the panel. The panel is the waiter: it takes the order and presents the result. The core agent is the head chef: it breaks the order into tickets, gives each ticket to a specialist worker, verifies the worker's handoff, and plates the final answer.

4.1 The parts

Part	Job	Where it lives
Panel (waiter)	Chat box, live activity strip, dock and telemetry, approval cards, CONVERSE and voice-switch buttons, file attach	panel.py, Flask at 127.0.0.1:5000
Core agent (head chef)	Plan tickets, delegate, verify handoffs, plate answers, enforce approval tiers, stop repeated failing actions	agent.py, run_agent_turn and plan_tickets
Specialist workers	Researcher, coder, and other roles, each with a pure-prompt skill pack. A worker runs as a separate agent.py job and writes a handoff file	skills/*.md, jobs/queue and jobs/done
Model dock	List of models with roles, VRAM cost, and Ollama tags. The core reads dock/config.json for the main brain	dock/config.json, dock/manifests/*.json
Memory	One-line index plus one file per fact, plus a SQLite episode log and JSONL chat transcripts	memory/, memory.db, chats/*.jsonl
Browser, two kinds	User browser: the visible desktop browser the user controls, the agent cannot click inside it. Agent browser: headless Playwright Chromium the agent controls	agent.py browser tools, open_visible_browser vs browser_open
Voice sidecar	Separate local server for ears and mouth, so voice never takes VRAM from the brain	voice-sidecar folder, not in this archive, see Chapter 9

4.2 Why orchestration, and why it is hard

A single 14-billion-parameter model cannot hold a long system prompt, a long history, a large tool list, and a hard task at the same time without losing track. Splitting the work lets each worker carry a small prompt and a small tool set. The cost is coordination: tickets must be clear, handoffs must carry evidence, and the head chef must actually check the evidence instead of trusting a worker's claim. Much of the later bug fixing in this project was coordination fixing, not model fixing: duplicate guards, job safety, anaphoric orders, plate checks, and attribution of panel rows.

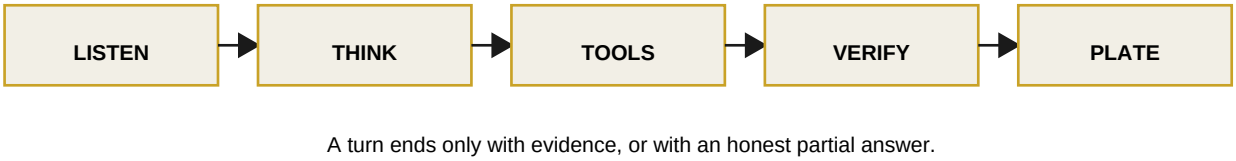


Figure 3. A single turn. Listen, think, use tools, verify, plate. If verification fails, the answer must say so, or say it is partial, rather than inventing a confident finish.

5. What is in the archive, file by file

The uploaded archive, agent-lab-2.zip, contains 328 files. It excludes two large folders that were too big to upload: the Python virtual environment (.venv) and the voice-sidecar folder. Everything else that mattered for a rebuild is in the archive, including the final agent and panel files, the model dock, the memory system, the chat history, and the job handoffs. If you rebuild from this archive, you will need to recreate the venv and the sidecar from Chapter 8. Do not assume they are hidden inside the zip.

Path in archive	What it is	Count / size in archive
agent.py	The core agent, final state. 9,538 lines, compiles with py_compile. Includes voice, jobs, browser, dock, memory, and security scan.	1 file, 443,256 bytes
panel.py	The Flask panel, final state. Footer reads V22.9.2. Includes CONVERSE, voice-switch, and PCM mic capture (Chapter 9).	1 file, 95,727 bytes, 1,230 lines
panel-actual.py, panel-clean-backup.py	Older panel backups. Do not run them by accident. Use panel.py.	2 files
agent_backup_Lab11.py, agent-backup-*.py	Older agent backups from Lab 11 and October 14. Useful for diffing, not for running.	3 files
dock/config.json	Main brain setting. At the end it reads huihui_ai/qwen3-abliterated:8b, left from a test.	1 file
dock/manifests/*.json	Model cards for qwen3:14b, qwen3:4b, and dolphin3:8b, with VRAM and role.	3 files
skills/*.md	Pure-prompt skill packs for workers: coder, researcher, security review, browser operator, window control, and others. Skills are prompts, not auto-posting code.	41 files

Path in archive	What it is	Count / size in archive
memory/, memory.db	File-per-fact memory plus a SQLite episode log. Episodes table: 2,323 rows in the archive copy.	Many files, 1,355,776 bytes for memory.db
chats/*.jsonl	One JSON line per turn, user prompt plus final answer only, text truncated to 4,000 characters. Tool-level detail is in memory.db, not in these files.	50 files, Oct 11 to Oct 29, 2026
jobs/done/*.md	Worker handoff reports, one per completed job brief.	80 files
jobs/running/*.log, *.json	Worker run logs. Many correspond to completed jobs and were never cleaned up. Do not read a running log as proof a job is still running.	90 files
voice/	Voice samples and state. reference-22.wav, reference-28.wav and reference-28-long.wav, the original demo MP3s, reference-28.txt, and active-voice.json with slot 28.	8 files
research/, notes/	Short research notes, including Ollama on Windows and TTS tools. The TTS note in the archive is a stub, not a full report.	4 files
start-agent.bat	Windows starter: change to E:\AGENT\agent-lab, activate .venv, run python agent.py, pause.	1 file, 6 lines
hidden_files/	Decider log and a native-transport 400 log. Diagnostic material, not user documentation.	2 files
screenshots/, uploads/	Empty folders in the archive. They were placeholders on the PC.	Empty

Memory warning. Two files in the archive need a warning, not a reading. The agent's own memory folder contains old, duplicate, and in some cases wrong fact files, and a cleanup was pending but never completed. Treat memory/facts as a record of what the agent once believed, not as a verified source. If you rebuild, start memory clean, or audit it first, and do not copy sensitive or personal notes from the archive into a public repository without checking them first. This book deliberately does not reproduce private memory notes.

5.1 The final versions, and how to prove them

Install proof in this project was always the footer version string, after a panel restart and a hard refresh (Ctrl+Shift+R in the browser). If the footer does not read the version you installed, stop and fix the install before you debug anything else. The archive's panel footer reads V22.9.2, and its agent file is 9,538 lines, matching the v14.9.1 agent build. The panel header comment still says v22.7 in its first line, because that line was never updated when later voice work was added. Trust the footer, not the header comment.

Build	Agent	Panel	Note
v14.8.14 + v22.8	8,925 lines	1,080 lines	Fixes G1 to G8: delete, jobs, anaphoric orders, plate check, chatter, browser boundary, monitor, row attribution.

Build	Agent	Panel	Note
v14.9 + v22.9	9,410 lines	1,189 lines	Voice converse mode v1, preset then cloned voice path.
v14.9.1 + v22.9.1	9,538 lines	1,229 lines	Voice-switch toggle, slots 22 and 28, active slot persisted.
Final in archive	9,538 lines	1,230 lines, footer V22.9.2	Panel PCM mic capture, replacing MediaRecorder webm. Agent unchanged from v14.9.1.

Battery numbers quoted in build notes, for example 256, 273, or 291 tests passing, were the builder's sandbox runs with mocked audio and sidecars. They prove the code compiled and the logic held in a sandbox. They do not prove live behaviour on Fahim's PC. This distinction matters, and it is repeated in Chapter 10.

6. Timeline: from first lab to final night

Dates below come from file timestamps and chat logs in the archive, and from the build history Fahim kept during the work. Where a date is a file timestamp, it is the last modification in the archive, not necessarily the day the idea was first discussed. The timeline is a reconstruction, not a lab journal, because no single lab journal file is in the archive.

When	What happened
6 Oct 2026	Project starts. Fully-local assistant on Windows, RTX 4070 SUPER, i7-14700K, 32 GB RAM. Project at E:\AGENT\agent-lab, models at E:\AGENT\ollama-models. Labs 1 to 3 completed early. Standing lessons: models invent excuses, verify against evidence, restart after every code edit.
7 to 11 Oct 2026	Early agent, memory redesign ideas, chat history, and the Diet: shrinking the system prompt from 7,677 to 2,658 characters and the tool set to 7 core tools, with specialists on demand.
12 Oct 2026	Head-chef directive. The agent must become an orchestrator, not a monolith. Strands Decider 2B assessed as a local decision model, after paid cloud routing was refused. Agent Reach noted as a researcher toolkit pattern.
14 Oct 2026	Orchestrator Lab 12 built: plan shown once, sequential workers, head chef verifies handoffs. Accuracy-first rule stated plainly. Many job handoffs in the archive are from this period, 14 October.
16 Oct 2026	Live-test night. Panel v21 BREACH TERMINAL becomes the favourite look. Approval rule fixed: ask only for PowerShell or command-line use and for serious writes. No-hardcoding rule fixed. No naming the agent, no hardcoded name. Tailscale dropped the same night.
19 to 23 Oct 2026	Agent testing and job delegation continued, with completed handoffs accumulating in jobs/done. No new agent release is recorded in the archive for this window.
29 Oct 2026, daytime	Image generation is ditched. Frontier models handle images better, so the agent-side image build is cancelled, not parked. Focus moves to voice. Voice converse scope is built as v14.9 and v22.9, then the voice-switch toggle as v14.9.1 and v22.9.1.

When	What happened
29 Oct 2026, evening	Voice cloning tests. Qwen3-TTS 0.6B in ICL mode with a long reference reaches about 70 percent for voice 28, beating 1.7B in a direct A/B. Fish Audio is chosen as the next test, then shelved. The live install begins.
29 Oct 2026, night, 21:00 to 21:52	Live voice install and debugging, in order: sidecar health passes, VAD 500 errors, torchaudio loader bug found and fixed with soundfile, webm fragment bug found and fixed with PCM capture, silent mic found to be a wrong browser mic input, latency 10 to 12 seconds, voice lines overlapping, conversation slowing as history grows. At 21:52 Fahim abandons the project. See Chapter 11.

7. How a turn works, step by step

7.1 A text turn

1. You type in the panel. The panel appends your message to the chat history, logs the episode, and starts a worker thread. 2. The core agent builds a system prompt from the current memory, tools, and dock state. 3. It calls the brain through Ollama. In the final build this is the native Ollama transport, /api/chat, with a fixed context size per model, not the OpenAI-compatible endpoint, so context can be observed. 4. If the brain calls a tool, the agent runs it, checks approval tiers, and feeds the result back. It stops after three identical-action retries, and nudges the model if it takes eight or more browser actions on one page without navigating. 5. When the brain finishes, the head chef checks the handoff: is there evidence for each claim, or is the answer partial? 6. The panel streams the answer, logs it, and returns to idle. The full tool-level trace goes to memory.db, episodes table. The chat transcript keeps only prompts and final answers.

7.2 A delegated job

1. The head chef writes a job brief as a JSON file in jobs/queue. The brief carries the goal, the skill pack, the original order verbatim, scoped memories, and an auto-approve list. 2. A worker runs as python agent.py --job jobs/queue/<id>.json, with no interactive prompts. 3. The worker writes a handoff report to jobs/done/<id>.md, ending with a Files written section that must be verified by listing files, not claimed from memory. 4. The head chef reads the handoff, checks the files, and plates only what is verified. A job that claims a file it never wrote is a failure, even if the rest of the text looks good.

7.3 A browser turn, and the two browsers

There are two browsers, and confusing them caused real bugs. The user's browser is the visible desktop browser you control. The agent can open it for you, with open_visible_browser, but it cannot click or read inside it. The agent's browser is headless Playwright Chromium, controlled by browser_open, browser_click, browser_type, and browser_js. Browser typing and JavaScript always ask for approval first. A visible-browser order, like open a site and show a section, must call the visible-browser tool and stop with a handover, not silently switch to the headless browser.

7.4 Memory, in plain terms

Memory is two things. First, a file-per-fact store under memory/facts, with a one-line index in MEMORY.md. Second, an episode log in memory.db and chat transcripts in chats/*.jsonl. When the agent needs a fact, it recalls scoped memories for the task, at most a handful, rather than loading the whole store into a small context. When it learns a fact, it saves it as a fact file. The system worked, but it accumulated duplicates and stale facts, and a cleanup was pending at the end. If you reuse this memory, audit it first.

7.5 Approval tiers, in plain terms

Tier Never asks: reads, typed non-destructive tools, and the action the user explicitly ordered. Tier Ask once per run: write_file, jobs, delegation. Tier Always asks: run_shell, delete_file, browser clicks and typing, and plating a result that involves a serious change. Approval is given through an Approve card in the panel, never by typing yes in prose alone in a way that the agent treats as a card. After one yes, up to five Tier-1 calls could auto-approve, then it asked again. That cap was Fahim's choice.

8. How to copy this work (reproduction guide)

Before you start. Read this whole chapter before you run anything. The archive is a snapshot, not an installer. There is no requirements file, no license file, and no README in the archive. You will need to create the venv and the voice sidecar yourself, and you will need to check every step live, because sandbox proof is not live proof.

8.1 Prerequisites

- Windows 10 or 11, 64-bit, with PowerShell. The build was Windows-first. Linux paths in guides are for the voice sidecar research only, not for the main agent.
- Ollama installed and running on http://localhost:11434. Set OLLAMA_MODELS to your model folder before you pull anything, so models do not fill your system drive.
- Python 3.13, available as python in PowerShell. The archive was built with Python 3.13, as shown by __pycache__/agent.cpython-313.pyc.
- A GPU with enough VRAM for qwen3:14b, about 9.3 GB for weights alone, plus KV cache. On a 12 GB card, 16K context is tight. On a smaller card, start with a smaller brain and a smaller context, and expect slower, less reliable tool calling.
- ffmpeg, for voice webm decoding in older builds and for converting demo MP3s to WAV. The final panel uses PCM, but ffmpeg is still useful for sample preparation.
- Chrome or Chromium, for the visible browser path, and Playwright's bundled Chromium as a fallback for the agent browser.

8.2 Step 1: Folders and models

Create the folders first, then pull models, then copy code. Do not copy code first and hope the models appear.

```
mkdir E:\AGENT\agent-lab
mkdir E:\AGENT\ollama-models
setx OLLAMA_MODELS "E:\AGENT\ollama-models"
```

Restart PowerShell after setx, so the new variable is picked up. Then pull, one model at a time, in the background if you like, because a 14B pull takes a while:

```
ollama pull qwen3:14b
ollama pull qwen3-vl:8b-instruct
ollama pull huihui_ai/qwen3-abliterated:8b
```

The researcher specialist in the archive history is MFDoom/deepseek-r1-tool-calling:14b. Pull it only if you plan to use researcher delegation. The challenger brain mentioned in later research is qwen3:30b-a3b, with the explicit tag, not the short qwen3:30b tag, which can drift. Do not make the challenger your default until you have tested it with the same suite you use for qwen3:14b.

8.3 Step 2: Python environment

The archive has no .venv and no requirements file. Create a fresh venv inside the project folder, activate it, and install only what the code imports. The agent imports, among others: openai, flask, and Playwright for the browser path. The panel imports flask. The voice sidecar, in Chapter 9, has its own venv and its own installs. Keep them separate. Do not install the sidecar packages into the agent venv, and do not install the agent packages into the sidecar venv, unless you have a reason and you record it.

```
cd E:\AGENT\agent-lab
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install openai flask playwright
playwright install chromium
```

If a tool is missing at runtime, the agent will tell you honestly, for example that no PDF library is installed, rather than inventing a result. Install the missing library in the venv, then ask again. Do not paste secrets, API keys, or passwords into chat to make an install work. Use the secure entry flow for any website login, and never store a key in memory or in a chat transcript.

8.4 Step 3: Copy the code and set the brain

Copy agent.py and panel.py from this archive into E:\AGENT\agent-lab, keeping those exact filenames. Copy dock/config.json into E:\AGENT\agent-lab\dock\config.json, and dock/manifests into the same dock folder. Then set the brain you actually want. The archive's config is left on huihui_ai/qwen3-abliterated:8b from a test. If you want the default orchestrator brain, change it back:

```
notepad E:\AGENT\agent-lab\dock\config.json
```

Change main_brain to qwen3:14b, save, and restart the agent and the panel. Do not judge the agent on the abliterated brain unless you are specifically testing that brain. It is weaker at tool calling, by design of the test, and the malware refusal still applies in code before the model.

8.5 Step 4: Run the panel and prove the install

```
cd E:\AGENT\agent-lab
.\.venv\Scripts\Activate.ps1
python panel.py
```

Open http://127.0.0.1:5000 in your browser. Hard refresh with Ctrl+Shift+R. Check the footer first. It must read V22.9.2 for the archive files. If it reads an older version, you are running an older panel file, or the browser cached the old page. Stop there and fix the file and the cache before you debug anything else. Then check that ollama ps shows qwen3:14b loaded, if that is your chosen brain, and that the panel dock shows the same model.

```
ollama ps
```

8.6 Step 5: First live tests, one per turn

Run one test per turn and wait for the turn to fully finish before you type the next prompt. Overlapping turns were a known fragility in this build: sending prompts while the agent was still answering made later oddities harder to attribute, and could interleave turns in the panel. Start with these, in order:

- Brain question: Ask a simple factual question with no tools. The answer should be clean, with no brain echo and no invented detail.
- File question: Ask the agent to list files in the project folder, then to read a small known file. Check that the file it claims to have read actually exists.
- Visible browser: Ask it to open a well-known site in your visible browser. It should open it and stop with a handover, not claim to have clicked inside your browser.

- Headless browser: Ask it to open a page in its own browser and report what it sees. Check the report against the live page.
- Approval: Ask it to run a shell command. It must ask for approval through a card first. Do not accept prose approval as a substitute in your own tests.
- Destructive order: Ask it to delete a test file you created for that purpose only. Check that the deletion is verified, not just claimed, and that a failed deletion is reported as a failure, not hidden.

If a test fails, save the exact error text, the footer version, the brain in dock/config.json, and whether you waited for the previous turn to finish. Those four facts make a failure diagnosable. Without them, it is just a story.

8.7 Step 6: Context size

The archive files carry NATIVE_NUM_CTX = 8192. Fahim later ran at 16384 and reported speed felt fine. If you want 16K, set it in one place and record how you set it, slider or file edit, because a reinstall will reset a file edit to 8192. Do not assume 16K fixes confabulation. A bigger window keeps more history, which can preserve confusing history as well as useful history. If later turns get slower, context growth is the first suspect, not the model alone.

Missing pieces. What is NOT in the archive, and why that matters. No .venv, no voice-sidecar code, no requirements file, no license, no README, no lab journal, and no 28-test suite file. The voice sidecar must be recreated from Chapter 9. The test suite must be written from the principles in Section 8.6. If you publish this repository, add a license file of your own choosing and a README, like the one delivered with this book, and do not claim the archive contains files it does not.

9. The voice lab: the last big push

Voice was the final focus, after image generation was ditched on 29 October 2026 because frontier models handle images better. The goal was not push-to-talk. It was a conversing mode, flipped on by a toggle, where the assistant talks, responds, and lets you interrupt it, like a person in conversation. Wake-word was deferred as an optional way to flip that toggle later, not as the first step.

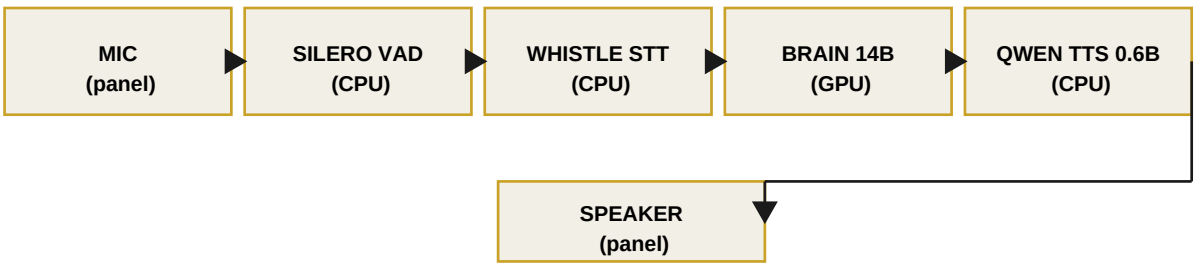


Figure 4. Voice v1 cascade. Panel microphone to Silero voice activity detection, to Whistle speech recognition, to the unchanged 14B brain, to Qwen3-TTS on CPU, to the panel speaker. The brain stays on the GPU. Voice never takes VRAM.

9.1 The sidecar contract

The agent talks to a separate local server, the voice sidecar, at http://127.0.0.1:8002, localhost only, never exposed beyond the PC. The contract the agent expects is:

Endpoint	Input	Output
GET /health	None	ok true, plus vad, whistle, faster_whisper, qwen_tts, kitten true or false, with error text for each
POST /vad	audio_b64, sample_rate, format pcm16 or webm	speech true or false. Any error counts as no speech, fail-closed, never a phantom interruption.
POST /transcribe	Same as /vad	text, from Whistle first, Faster-Whisper as fallback
POST /tts	text, voice clone or preset, ref_audio path, ref_text transcript	WAV bytes, HTTP 200. Clone failure retries once as preset, with an honest indicator.

Panel mic chunks in the final panel arrive as format pcm16, downsampled to 16000 in the browser, flushed every 500 milliseconds. Direct agent calls use format pcm16. Older builds used webm, and that difference is the subject of Section 9.4.

9.2 Voices 22 and 28, and the 70 percent result

Voice and persona are separate layers. Voice is timbre, from a reference sample. Persona is character, prompt, and behaviour. Persona was not built in v14.9 or v14.9.1, and it is not in this archive as code. What was built was a switch between two voice slots, labelled 22 and 28 as Fahim referred to them.

Slot	Direction	Sample in archive
22	American enthusiastic tease, medium, bright, enthusiastic	reference-22.wav, 169,060 bytes, about 5.3 seconds, plus the original demo MP3
28, primary	Caitlin voice, teasing, the voice Fahim chose as the main voice	reference-28.wav, 360,292 bytes, the long version, plus reference-28-single.wav, the short version, and reference-28.txt

The long 28 reference was built by joining the two Caitlin demo clips, 28 and 21, into one 11.26-second WAV at 16 kHz mono. The long 22 reference, if you recreate it, joins four 22-family clips into 19.39 seconds. The active slot at the end of the archive is 28, in voice/active-voice.json. Each slot needs its own user-provided sample at install. A slot with no sample speaks with a preset voice and shows a dashed outline and a PRESET tag. It never pretends to be a clone it does not have.

Cloning quality, in order of testing on 29 October 2026: a 5-second single-line reference in x-vector mode sounded robotic. A long reference in x-vector mode still sounded robotic, with no emotion. ICL mode, with a Whistle transcript of the long reference, plus an expressive test sentence, was judged by Fahim as much better, about 70 percent there, up from robotic. The 1.7B model was then tried in the same ICL setup and did not beat the 0.6B model in his judgement. The decision, per the rule set before the test, was to stop chasing model size and lock in 0.6B. A single run varies, because TTS sampling is random, so that verdict is not proof that 1.7B is always worse, only that there was no reason to pay its cost for this voice.

Two other voice facts matter for anyone copying this. First, using a good generated output as the next reference does not improve timbre past the original reference. It is a copy of a copy, with no new information about the true voice, and chaining rounds degrades it. It can transfer style or delivery, for one round, and passing the original plus a good generation together as a reference list is a fair experiment, but do not chain it. Second, the expressive style Fahim wanted, natural pauses, fillers like hmm and umm,

drawn-out words, playful negations, and a coy, teasing, acting-cute quality he called nekami, comes from the text the brain writes, not from the clone alone. TTS renders the text it is given. That style is a persona-layer job, and it was not built.

9.3 Fish Audio, and why it was shelved

Fish Audio, Fish Speech, was the one serious alternative engine considered, because expressive cloning is its strength. Kitten-TTS 2 was cheap to try but low expected gain. Sesame CSM-1B and Moshi were skipped for this purpose, because they are dialogue or full-duplex legs, not drop-in clone TTS. No engine was promised to beat Qwen 0.6B ICL, and all of them share the same ceiling: cloning a synthetic demo can carry TTS artefacts over.

Official requirements checked on 29 October 2026: Drama 3 needs at least 48 GB of GPU, NVIDIA only, no CPU, so it was completely out. S2-Pro needs at least 24 GB on GPU, or CPU. Fahim's card has 12 GB, so S2-Pro on GPU was out, and GPU use would also have broken the rule that voice must not take VRAM from the brain. CPU was the only viable test path, and a 4-billion-parameter model on CPU would be much slower than Qwen 0.6B on CPU. The cloud Fish API was excluded by the zero-cost veto, because it needs an API key. Fahim chose to shelve Fish rather than run the slow CPU test, and locked in Qwen 0.6B instead. Do not revive Fish in this repository without reopening that decision and accepting the CPU cost.

9.4 Three voice bugs, found live, in order

These bugs were not found in sandbox tests. They were found only when Fahim tried to talk live on 29 October 2026, and they are documented here in the order they were found, because the order matters: fixing one revealed the next.

Bug 1: torchaudio loader

Symptom: CONVERSE stayed on LISTENING, sidecar log showed POST /vad 500 on every chunk. Cause: silero_vad.read_audio calls torchaudio.load, and torchaudio 2.11.1 on CPU requires torchcodec for audio input and output. The earlier Silero test had only loaded the model, never read a WAV through that path, so the bug stayed hidden. Diagnostic: a test script posted a full webm file and a PCM file to /vad, and both returned the same torchaudio error, proving it was not ffmpeg. Fix in the final sidecar: read WAV files with soundfile, which was already installed for Qwen TTS, convert to a torch tensor, and call get_speech_timestamps with an explicit sample rate. Do not install torchcodec or pin torchaudio as the first fix. After the fix, the diagnostic returned /vad 200 with speech true for both formats.

Bug 2: webm fragments

Symptom: after Bug 1 was fixed, live /vad returned to 500, with a new detail in the console: ffmpeg webm decode failed, Invalid data found when processing input, on every live chunk after the first. Cause: the panel used MediaRecorder with a 500-millisecond timeslice. The first chunk carries the webm header. Later chunks are fragments with no header, and ffmpeg cannot open a standalone fragment. A sandbox test confirmed it: a header prefix decodes with return code 0, a middle fragment fails with the exact same error. Fix in the final panel, v22.9.2: capture PCM16 directly in the browser with AudioContext and ScriptProcessor, downsample to 16000, flush every 500 milliseconds as an independent PCM chunk. The sidecar PCM path was already verified by the diagnostic. The agent file did not change for this fix.

Bug 3: silent mic

Symptom: after Bug 2 was fixed, sidecar showed no errors, panel showed POST /api/voice-audio 200 every 500 milliseconds, but the panel stayed on LISTENING and never responded. Diagnostic: a new sidecar log line printed VAD speech, maximum amplitude, RMS, sample count, and sample rate per chunk. Every chunk showed speech False, max 0.000, RMS 0.000, about 7,843 samples at 16000. Format was correct, content was digital silence, not quiet speech and not a Silero threshold miss. Cause, found by Fahim: the browser tab mic setting was picking the wrong mic input. Fix: change the mic input in the

browser tab settings, and check Windows Sound input level and Windows Voice Recorder playback before blaming the code again. If Voice Recorder hears you but the panel still shows max 0.000, then the panel capture path is at fault. If Voice Recorder is silent too, the operating system or device is at fault.

9.5 Latency, overlap, and slowing down

Once the mic was correct, voice did work, but the first spoken reply took about 10 to 12 seconds, and the experience degraded as the conversation went on. Three mechanisms explain that pattern, and they are different fixes, so do not treat them as one slowdown.

- History growth: every turn re-reads the full chat history. At 16K context, later turns carry more tokens than early turns, so generation gets slower over a conversation. Trimming or summarising voice history is a separate fix from making a single turn faster.
- CPU TTS queue: Qwen TTS on CPU generates a full sentence WAV before the panel plays it, in non-streaming mode per sentence. If generation is slower than playback, the queue backs up, and later sentences arrive late, even if the brain was fast.
- Playback race: in the final panel, voicePlaying is only set after the /api/voice-tts fetch completes. If a second sentence arrives during that fetch, it can start its own playback, and two voice lines run over each other. The fix is to mark playback as busy before the fetch, not after, and to serialize the queue strictly. That race was identified but not fixed in the archive, because the project stopped that night.

A think block, if enabled, also delays speech, because the streaming code holds back think text and only speaks complete sentences after the think block closes. For voice turns, disabling think, or speaking only after a short first sentence, is a trade-off to test, not a free win: you lose reasoning depth for speed.

9.6 Recreating the sidecar, since it is not in the archive

Create a separate folder, for example E:\AGENT\agent-lab\voice-sidecar, with its own venv. Do not install the sidecar into the agent venv. Install CPU torch first, so later installs do not pull the large CUDA torch, then silero-vad, FastAPI, uvicorn, soundfile, numpy, cactus-needle for Whistle, faster-whisper as a fallback, and qwen-tts 0.1.1. Do not install flash-attn on CPU, and do not install Kitten unless you need the preset fallback. The Qwen 0.1.1 API in the archive history is Qwen3TTSModel.from_pretrained, then generate_voice_clone, with x_vector_only_mode True when no transcript is given, and False with a transcript for ICL mode. ICL mode requires ref_text at runtime, or it raises a ValueError. The sidecar must use the ref_audio path from each /tts request, not a hardcoded sample, or the voice switch will silently change nothing.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install torch --index-url https://download.pytorch.org/whl/cpu
pip install silero-vad fastapi uvicorn soundfile numpy cactus-needle faster-whisper
pip install -U qwen-tts
uvicorn voice_sidecar_starter:app --host 127.0.0.1 --port 8002
```

Check health with curl.exe http://127.0.0.1:8002/health. Expect vad, whistle, faster_whisper, and qwen_tts true, and kitten false if you did not install Kitten. That last false is expected for cloning. Leave the sidecar window open while you use voice. Closing it stops voice mode, and the panel must fall back to text with an honest message, never a silent turn.

10. What worked, what did not, with evidence

10.1 What worked, in the archive

- Brain question and answer, clean, with no brain echo, in live tests on 29 October 2026.

- Desktop control: Discord launched with a verified process ID, Chrome private mode opened, brain switching landed, in the same live session.
- Honest failure: a PDF question answered with a plain no-library error, instead of an invented PDF summary. That honesty is a feature, not a bug, in this project.
- Job delegation: 80 completed job handoffs in jobs/done, with skill packs, verification sections, and Files written lists, in the archive.
- Voice pipeline, in pieces: Silero test passed, Whistle transcribed a reference clip with core words exact, Qwen 0.6B basic clone generated a WAV, sidecar health passed, diagnostic VAD passed after the soundfile fix, PCM capture arrived every 500 milliseconds with no sidecar error.
- Voice clone, in a test: ICL mode with a long reference reached about 70 percent for voice 28, in Fahim's judgement, up from robotic x-vector output.

10.2 What did not work, in the archive

- Free-chat confabulation: the agent invented opinions and facts instead of saying it did not know, especially on entity and show questions. See the specimens below.
- Anaphoric and context loss: follow-up questions lost their referent. See Halloween below.
- Canned chatter fallbacks: vague prompts received Glad it worked or You are awake, what do you need, instead of a clarification or an honest I do not know.
- Voice conversation, as a whole: 10 to 12 seconds to first spoken reply, slowing over a conversation, and overlapping voice lines. Not usable as conversation, in Fahim's verdict on 29 October 2026.
- Voice install fragility: three live bugs in one night, torchaudio loader, webm fragments, and a wrong mic input, each hiding the next. Sandbox tests did not catch them.
- Unverified base: the 28-test suite was declined on 29 October 2026, so the v14.8.14 fixes and everything built on them stayed sandbox-only. Later oddities were therefore harder to attribute.

10.3 Three failure specimens, short and verifiable

Specimen 1, Edgerunners, 29 October 2026, about 17:01, version and footer unconfirmed. The agent was asked about Cyberpunk Edgerunners Season 2, never named a verifiable character, episode, or plot point, gave a generic invented opinion instead of saying it did not know, answered a name question with a greeting honorific, and only on the final entity question gave the honest answer that it had no name stored. Pattern: free-chat invention plus history slot-fill, no verification gate on opinion or knowledge turns.

Specimen 2, Halloween, 29 October 2026, 21:21 to 21:22, typed, huihui brain. Question: when is Halloween this year. Answer: October 31. Follow-up: what day is it. The agent called get_time and answered Thursday, 29 October, which is today, not Halloween's weekday. Correct answer, verified by calendar: Saturday, 31 October 2026. Follow-ups received canned fallbacks and repeated get_time calls. Pattern: anaphoric referent loss, then chatter fallback instead of clarification.

Specimen 3, last chat in the archive, 29 October 2026, 21:34 to 21:38. A greeting was answered with a greeting honorific as if it were a name. A question about the plot of Darling in the Franxx, misspelled in the prompt, was answered with an invented plot about a woman named Darling and a man named Frank, with themes of love, betrayal, and self-discovery, none of which is the actual show. A vague follow-up was answered with I have no memories related to whirls, then the invented plot continued. Pattern: same family as Specimen 1, on the final night, on the final brain left in config.

Why specimens matter. These specimens are included because a future builder needs to see the failure mode, not just a summary of it. They are short, they are from the archive's own chat files, and they are not presented as a complete test suite. Do not use them to mock the author. Use them to test whether your fix actually changes the behaviour on a new, similar question, not just on these three prompts.

11. Where it stopped, in the author's own terms

On the night of 29 October 2026, after the voice tests, Fahim said he was abandoning this project, not parking it. He had tried to make a great thing at a smaller scale, and he felt he had failed, with no real result. He said he felt powerless when nothing worked.

That sentence is kept here plainly, because it is the honest ending of this project, and because a future reader who finds this repository should not have to guess why it stops. The project did not stop because a single file was missing. It stopped because the whole that Fahim wanted, a local agent he could talk to like a person, instantly and properly, did not work as a whole, even after the pieces were fixed one by one that night: mic input, VAD loader, webm fragments, silent PCM, latency, overlap, and slowing history.

In the author's own terms, as he put it at the end:

> *I will wait for a better AI agent or bot to help do this, so that people can have their own local agent in the future. A true AI companion that is truly intelligent.*

That is not a roadmap and not a promise. It is a hope, left in the open, for whoever picks up this kind of work next, with better tools, better models, or a better way to do conversation locally. If you are that person, this book is for you. Take what is useful, leave what is not, and test live, because that is where this project lived and where it stopped.

So long, and goodbye, from Fahim. The repository stays, as he asked, so the work does not disappear with the decision to stop.

12. If you pick this up: what to do differently

Do not start by adding features. Start by proving one daily job live, end to end, with a version footer, a real run, and a real result, and keep that job working while you add the next thing. That was the missing bar in this project, and it is the first thing a future builder should restore.

- Fix the playback race first, before any voice tuning: mark playback as busy before the TTS fetch, not after, and serialize the queue strictly. Overlap destroys trust faster than latency does.
- Decide think for voice explicitly: if think is on, measure how long it delays the first speakable sentence, and decide if that cost is worth it for a voice turn. Do not leave it as an accidental default.
- Trim voice history: do not send the full chat history to every voice turn. Summarise, window, or separate voice history from text history, and measure later-turn latency, not just first-turn latency.
- Consider a speech-native or GPU TTS path honestly: CPU TTS at 0.6B will always lag cloud speech. If conversational latency is your bar, you need a faster TTS path, a smaller and faster voice model, or a speech-native model, each with its own trade-offs in quality, VRAM, and reliability. Moshi was noted as an experimental open option, not a ready answer.
- Test live, one turn at a time, with the four facts from Chapter 8: footer version, brain, whether you waited for the previous turn, and the exact error text. Do not accept sandbox proof as live proof.
- Audit memory before you reuse it: remove duplicates and stale facts, or start clean. A memory that invents with confidence is worse than a memory that says it does not know.
- Keep the approval and safety rules: ask for shell and serious writes, refuse malware at the routing layer, keep secrets out of chat and out of memory, and do not hardcode conversational fixes. Those rules were not the reason voice failed, and dropping them will not make voice faster.
- Keep the zero-cost rule explicit if you keep it: if you add a paid or cloud step, say so in the README and in the code, so a future user is not surprised by a bill or by data leaving the machine.

Finally, keep the ending honest in your own repository. If you fork this work and it still does not work as conversation, say so in your README, with the same plainness as Chapter 11. A future reader will trust you more for it, and will waste less time repeating the same night.

Appendix A. File inventory and counts

Counts were taken from the extracted archive, agent-lab-2.zip, on the build machine. They describe the archive as delivered, not Fahim's PC at any other moment, and they exclude .venv and voice-sidecar, which were not in the zip.

Group	Count
Total files in archive	328
agent.py lines	9,538
panel.py lines	1,230, footer V22.9.2
Skill packs in skills/	41
Chat transcripts in chats/	50, 11 Oct to 29 Oct 2026
Completed job handoffs in jobs/done/	80
Running logs in jobs/running/	90, many are old logs, not live jobs
Episodes in memory.db, episodes table	2,323
Voice files in voice/	8, including 2 demo MP3s, 3 reference WAVs, 1 transcript, 1 active-slot file
Model manifests in dock/manifests/	3

A note on the two database files in the archive: memory.db is the main episode log, 1,355,776 bytes. episodes.db is a small, early file with 3 episodes, from 8 October 2026. Do not merge them without checking which one the final agent actually writes to. In the final agent, init_db and log_episode use memory.db.

Appendix B. Tool families in agent.py

This is a grouped summary, not an exhaustive function list. Function names are given so you can find them in agent.py with a text search. If a function is not in your copy, you are not running the archive version.

Family	Examples in agent.py	Approval note
Time	get_time	No ask
Files	read_file, write_file, delete_file, list_files, read_pdf, list_code_files, read_code	Write and delete ask. Read does not.
Memory	save_memory, recall_memory, log_chat, log_episode, distill_session	No ask, except where a write is serious
Web	web_search, web_fetch, advisory_lookup	No ask for read. Advisory lookup is defensive only.
Shell, guarded	run_shell, with blocklist and allowlist, plus _is_blocked and _is_allowlisted	Always asks, Tier-2
Desktop and windows	launch_app, find_app, list_windows, focus_window, close_window, move_window, desktop_screenshot	Launch and close are serious actions. Screenshot is read-like, but check consent in your own use.

Family	Examples in agent.py	Approval note
Browser	browser_open, browser_click, browser_type, browser_js, browser_video, open_visible_browser, open_browser_private	Click, type, and JS always ask first.
Jobs and delegation	create_job, launch_job, job_status, read_handoff, delegate, plan_tickets, ticket_status, plate_answer	Create and delegate are Tier-1 or Tier-2 by the approval rule.
Dock and brains	dock_status, list_brains, switch_brain, switch_brain_tool, load_brain	Brain switch asks if a turn is running, to avoid unloading mid-inference.
Skills	use_skill, with skill packs in skills/*.md	Skills are prompts. Auto-posting is not a skill.
Security scan	scan_code, with Python AST checks and a Bandit runner where available	Defensive review only. Never use it to build malware.
Voice	voice_health, voice_tts, voice_vad, voice_transcribe, voice_slot_info, voice_switch, run_voice_turn, voice_session_feed	Voice turns use the same approval tiers as text turns. Voice alone never auto-approves a destructive action.
Decider, local routing	expedite, _decider_ask, _suggested_specialists	Local CPU routing helper, not a paid API. See the Decider note in Chapter 4.

Appendix C. Model dock

Model, Ollama tag	Role	VRAM in manifest
qwen3:14b	Main brain, default orchestrator, strong reasoning and reliable tool calling	9.3 GB
qwen3:4b	Quick answers, light and fast, later voice-loop candidate in early notes	2.7 GB
dolphin3:8b	Assistant, fast and efficient conversational model	3.1 GB
huihui_ai/qwen3-abliterated:8b	Uncensored test brain, weaker tool calling, left in dock/config.json at the end	Not in manifests, pulled separately
MFDoom/deepseek-r1-tool-calling:14b	Researcher specialist in build history	Not in manifests, pulled separately
qwen3-vl:8b-instruct	Vision specialist in build history	Not in manifests, pulled separately
qwen3:30b-a3b	Heavyweight challenger in later research, via Ollama auto-split, not default	Not in manifests, research only

VRAM numbers above are from the dock manifests, not measurements on Fahim's PC at run time. Ollama's actual VRAM use varies with context size, KV cache, and offloading. Check ollama ps while the model is mid-task, and read the CONTEXT column, before you trust a manifest number for a live decision.

Appendix D. Safety, secrets, and limits

- No secrets in this repository: never paste API keys, passwords, or one-time codes into chat, into a job brief, into memory, or into a public repository. The archive contains no working keys, and this book contains none. If you find a key in an old file, rotate it and remove it from history before you publish.
- No malware: do not use this code to write remote-code-execution, malware, or exploitation tools. The routing-layer refusal in the agent is part of the design, not a bug to work around. Defensive scanning, vulnerability lookup, and security review are allowed uses.
- Localhost only: panel at 127.0.0.1:5000 and sidecar at 127.0.0.1:8002 are not meant to be exposed to the internet. If you expose them, you expose file, shell, and browser tools to anyone who can reach them. Do not do that without authentication, and do not do that on the basis of this book.
- Audio privacy: voice turns in the final build log text transcripts only, by design. Audio is not stored by default. If you change that, say so in your README and get consent first.
- Licence: there is no licence file in the archive. This book does not choose a licence for Fahim. If you publish this repository, add a licence file of your own, or, if you are republishing Fahim's work with permission, use the licence he chooses. Until then, treat the code as all rights reserved, and do not assume you may reuse it commercially.
- Attribution: this book is by and about MD Fahim Nafis, October 2026. If you quote Chapter 11, attribute it to him, and do not edit his closing hope into a marketing claim.

End of documentation. Archive: agent-lab-2.zip, 328 files, final agent 9,538 lines, final panel footer V22.9.2, 29 October 2026. Author: MD Fahim Nafis. Status: archived, abandoned, left for the future.

Thank you for reading this far. If you build the local companion Fahim hoped for, build it honestly, test it live, and leave a record as plain as this one.